

⚡ 05. Mongoose Queries & Update Comparisons

🔍 Querying in Mongoose (Thennables)

Mongoose queries return **Thennables (Query Objects)** rather than native promises, enabling chaining and await resolution:

```
index.js - Query Methods

// 1. Array of matching documents
const allUsers = await User.find({ age: { $gt: 20 } });
// 2. Single matching document or null
const user = await User.findOne({ email: "amit@gmail.com" });
// 3. Find by primary key _id
const userById = await User.findById("65c8a1f2 ... ");
```

⚖️ Updates: updateOne vs findOneAndUpdate (Critical Difference!)

Method	Return Value	Use Case
User.updateOne() / updateMany()	Metadata only: { acknowledged: true, modifiedCount: 1 }	When only operation status is needed without rendering the document.
User.findOneAndUpdate() / findByIdAndUpdate()	Actual Document! (Default: old document; Pass { new: true } for updated doc)	When updated document is required immediately for API response or UI.

🗑️ Deletions: deleteOne vs findByIdAndDelete

Method	Return Value	Description
User.deleteOne()	{ acknowledged: true, deletedCount: 1 }	Returns deletion status count only.
User.findByIdAndDelete(id)	Deleted Document Object	Returns the deleted document (crucial for audit logs or undo actions).



Schema Validations & The runValidators Trap



Schema Validations in Depth

```
models/book.js — Advanced Schema Validations

const bookSchema = new mongoose.Schema({
  title: { type: String, required: [true, "Title is required!"], maxLength: 50 },
  price: { type: Number, min: [1, "Price must be at least ₹1"] },
  genre: { type: String, enum: ["fiction", "tech", "history"] },
  inStock: { type: Boolean, default: true }
});
```



THE runValidators TRAP (MOST COMMON BUG): By default, Mongoose validation executes ONLY on `.save()`! For updates, you MUST explicitly pass `{ runValidators: true }`, or invalid data will bypass validation!

```
index.js — findByIdAndUpdate with runValidators

// Without runValidators: true, negative prices (-500) would be saved!
const updatedBook = await Book.findByIdAndUpdate(
  id,
  { price: -500 },
  { new: true, runValidators: true } // ➡ Triggers schema validation error!
);
```

Validation Summary Checklist:

- `required: [true, "Custom message"]` → Field must be provided.
- `enum: ["val1", "val2"]` → Restricts values to specified list.
- `min / max` → Enforces numerical or date limits.